TicketFlow documento di progettazione

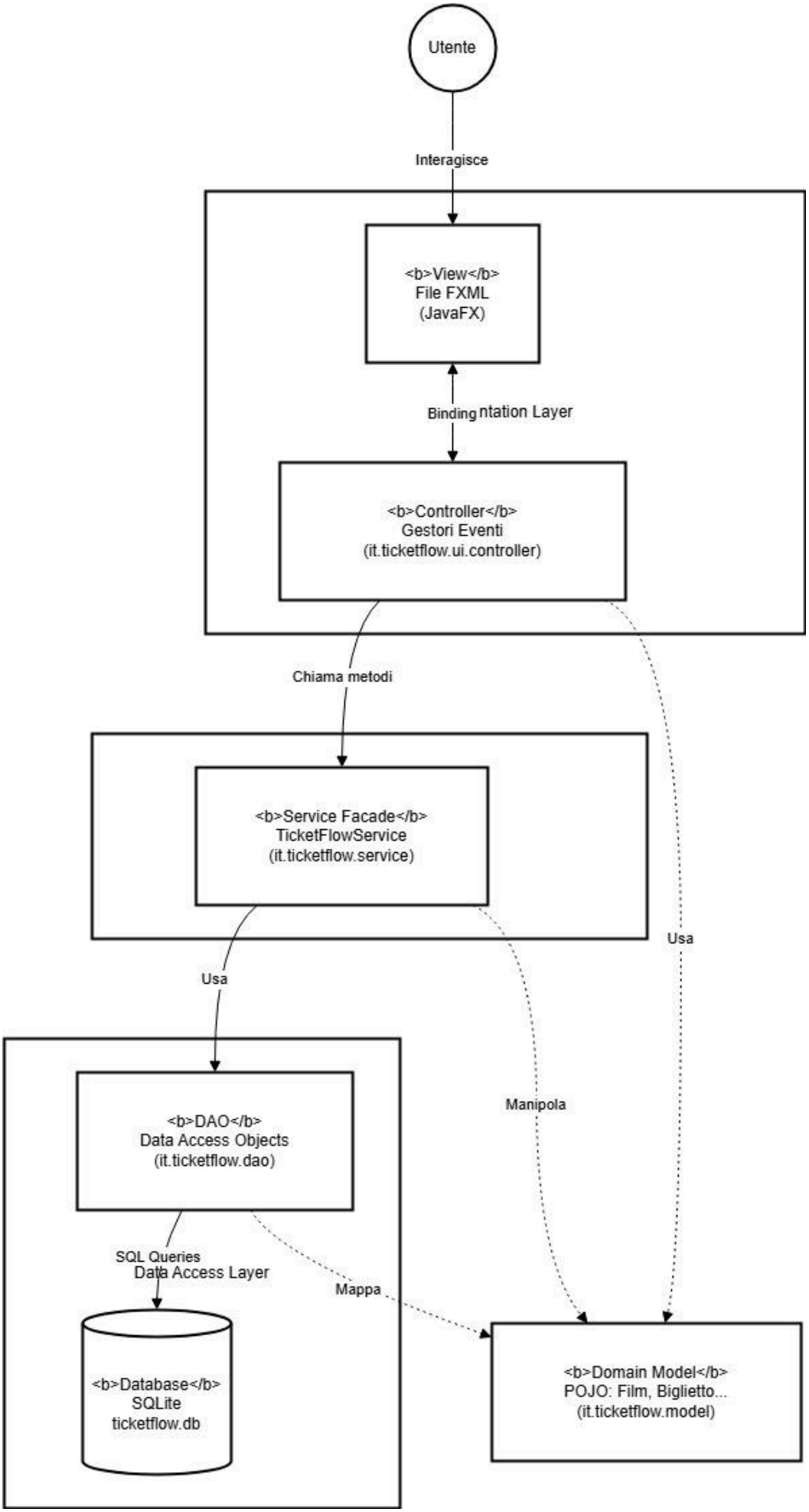
1. Diagramma architetturale

Scelta del Pattern Architetturale: Model-View-Controller (MVC)

Per la progettazione di TicketFlow, il nostro gruppo ha scelto di adottare il pattern architetturale **MVC (Model-View-Controller)**. Il nostro intento principale è definire fin da subito una netta separazione tra la logica di gestione dei dati e la loro rappresentazione grafica. In questa fase di design, riteniamo fondamentale disaccoppiare questi aspetti per garantire che lo sviluppo del backend possa procedere indipendentemente dall'interfaccia grafica. Questa scelta strategica mira a assicurare la massima modularità del sistema: il nostro obiettivo è facilitare la manutenzione futura del codice.

Sulla base di questa analisi, abbiamo progettato l'architettura del sistema come segue:

- **View (Livello di Presentazione):** Sarà il componente dedicato esclusivamente all'interfaccia grafica e all'interazione con l'utente. Intendiamo implementare questo livello utilizzando file **FXML (JavaFX)**, che definiranno la struttura visiva senza incorporare logica di business complessa. La comunicazione con il Controller avverrà tramite meccanismi di binding.
- **Controller (Livello di Controllo):** È progettato per fare da intermediario tra l'utente e il sistema. Qui implementeremo i **Gestori di Eventi**, il cui compito sarà intercettare le azioni sulla View e invocare i metodi pertinenti del livello di servizio, gestendo il flusso dell'applicazione.
- **Service Layer (Livello di Servizio):** Per mantenere pulito il Controller, è stato previsto l'inserimento di un **Service Facade**. L'intento è che il Controller deleghi a questo servizio l'esecuzione delle operazioni complesse e la manipolazione del **Domain Model**, isolando la logica applicativa.
- **Model (Livello Dati):** Rappresenterà lo stato del sistema e la logica di accesso ai dati. La nostra progettazione prevede due componenti distinti:
    - **Domain Model:** Classi **POJO** (come *Film* e *Biglietto*) che modelleranno le entità del dominio.
    - **DAO (Data Access Objects):** Il layer che si occuperà della persistenza, eseguendo le query SQL e mappando i risultati dal database SQLite agli oggetti del dominio.

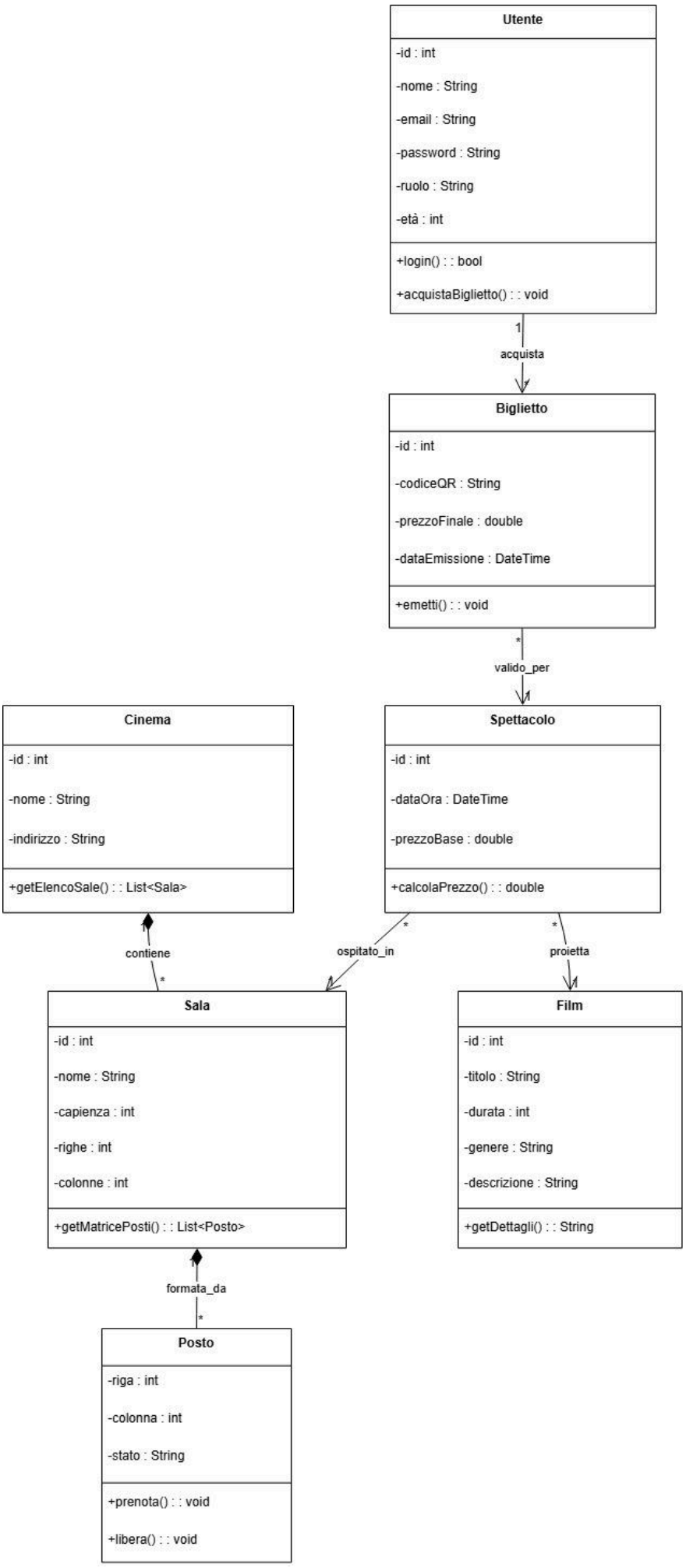

Utente
Interagisce
View
File FXML
(JavaFX)
Binding
ntation Layer
Controller
Gestori Eventi
(it.ticketflow.ui.controller)
Chiama metodi
Service Facade
TicketFlowService
(it.ticketflow.service)
Usa
Usa
Manipola
DAO
Data Access Objects
(it.ticketflow.dao)
SQL Queries
Data Access Layer
Mappa
Database
SQLite
ticketflow.db
Domain Model
POJO: Film, Biglietto...
(it.ticketflow.model)

2. Diagramma delle classi di dominio

Modello di dominio

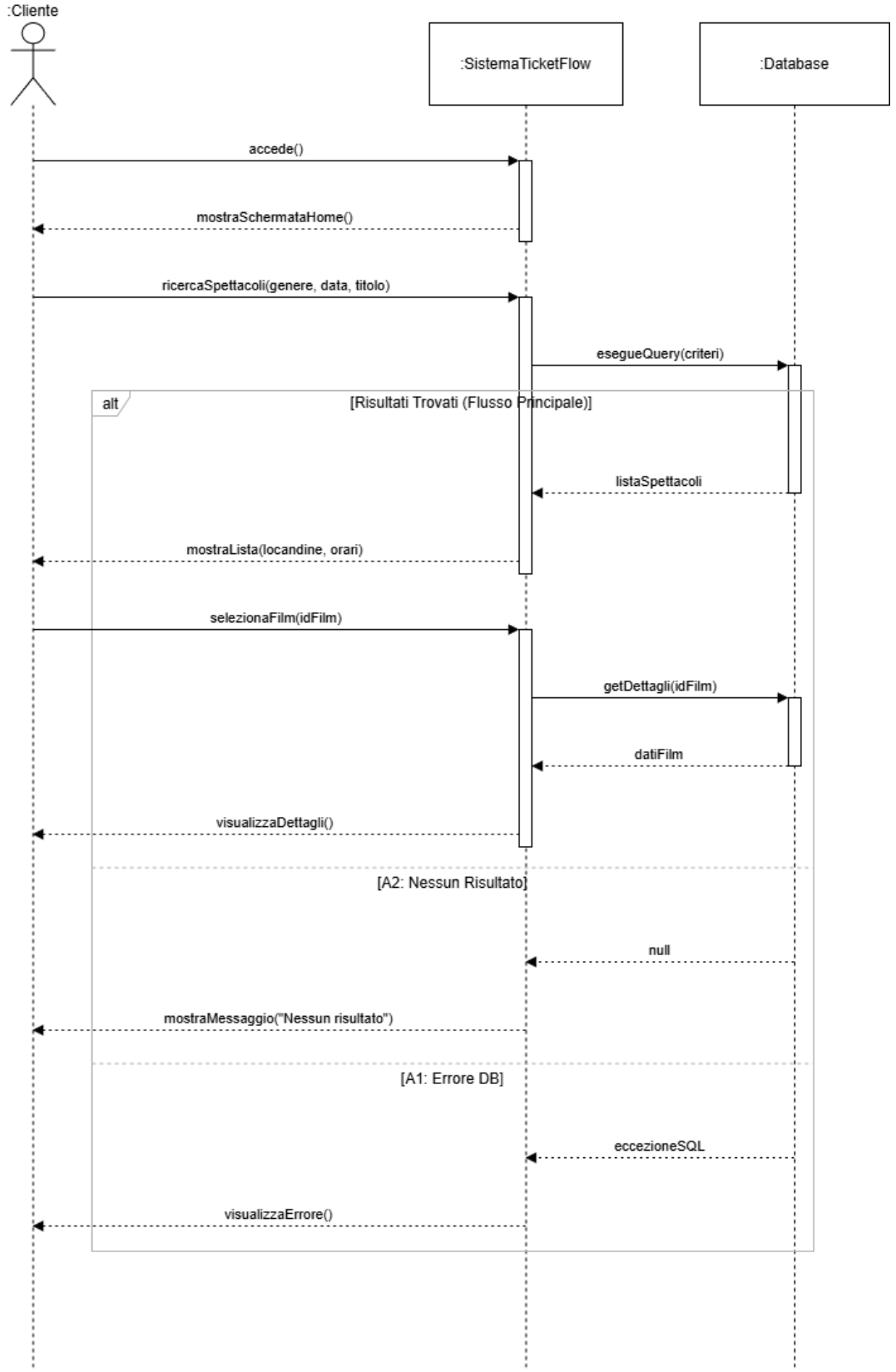
Classe	Attributi Principali	Responsabilità
Cinema	id, nome, indirizzo, elencoSale	Rappresenta la struttura fisica. Contiene le sale.
Sala	id, nome, capienza, righe, colonne	Rappresenta lo spazio fisico. Gestisce la matrice dei posti.
Posto	riga, colonna, stato (Libero/Occupato/Guasto)	Rappresenta la singola unità vendibile.
Film	id, titolo, durata, genere, descrizione	Contiene i metadati dell'opera cinematografica.
Spettacolo	id, dataOra, prezzoBase, sala, film	Associa un film a una sala in un tempo specifico.
Biglietto	id, codiceQR, prezzoFinale, dataEmissione, id_utente, id_spettacolo	Rappresenta il titolo di accesso venduto. Collega Utente e Spettacolo.

Utente	**id, nome, email, password, ruolo, età**	**Gestisce l'identità di Clienti, Manager e Super Admin.**

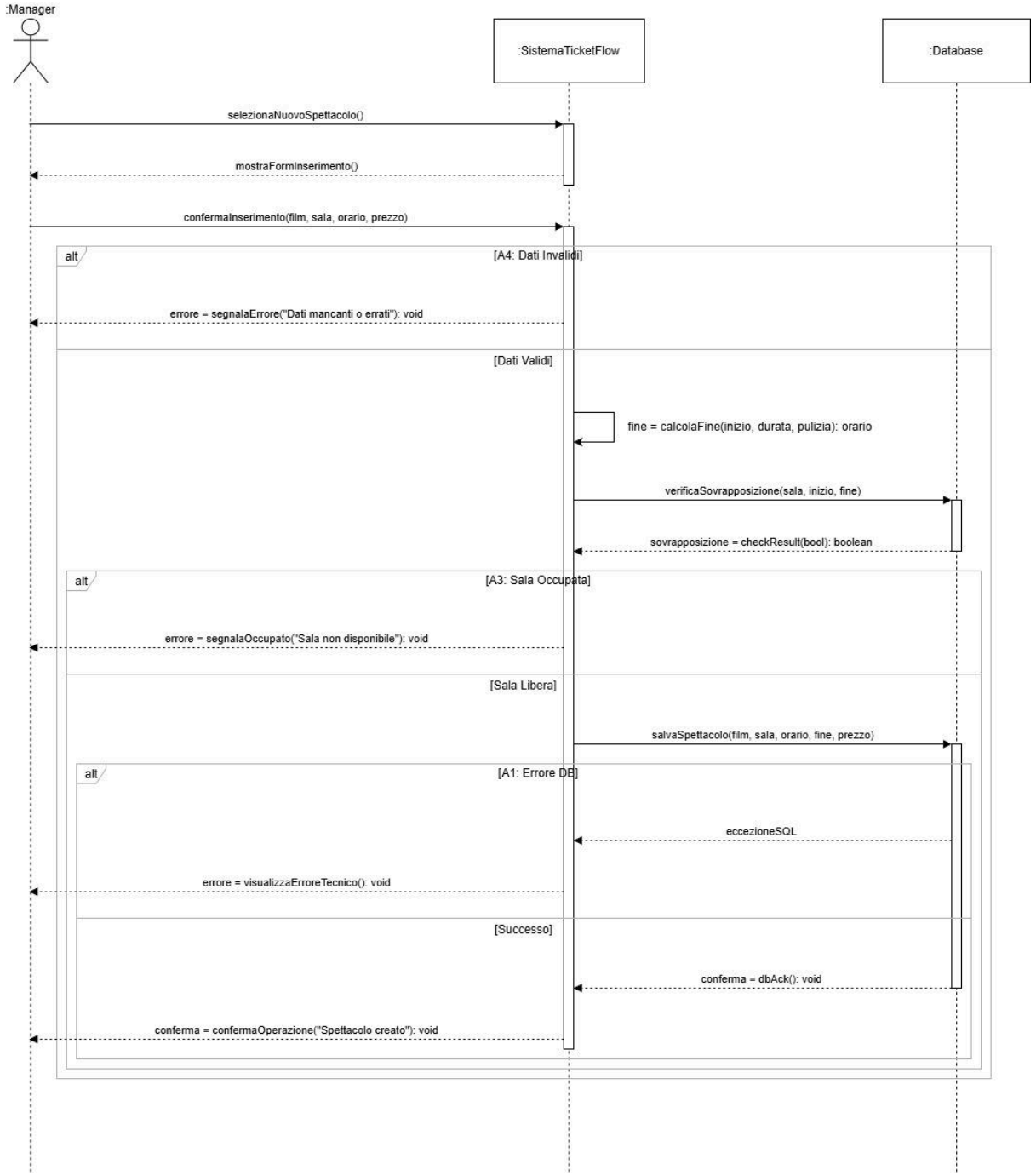

Utente
-id : int
-nome : String
-email : String
-password : String
-ruolo : String
-età : int
+login() : : bool
+acquistaBiglietto() : : void
1
acquista
*
Biglietto
-id : int
-codiceQR : String
-prezzoFinale : double
-dataEmissione : DateTime
+emetti() : : void
*
valido_per
1
Cinema
-id : int
-nome : String
-indirizzo : String
+getElencoSale() : : List<Sala>
Spettacolo
-id : int
-dataOra : DateTime
-prezzoBase : double
+calcolaPrezzo() : : double
1
contiene
*
*
ospitato_in
*
proietta
1
Sala
-id : int
-nome : String
-capienza : int
-righe : int
-colonne : int
+getMatricePosti() : : List<Posto>
Film
-id : int
-titolo : String
-durata : int
-genere : String
-descrizione : String
+getDettagli() : : String
1
formata_da
*
Posto
-riga : int
-colonna : int
-stato : String
+prenota() : : void
+libera() : : void

3. Diagramma di sequenza

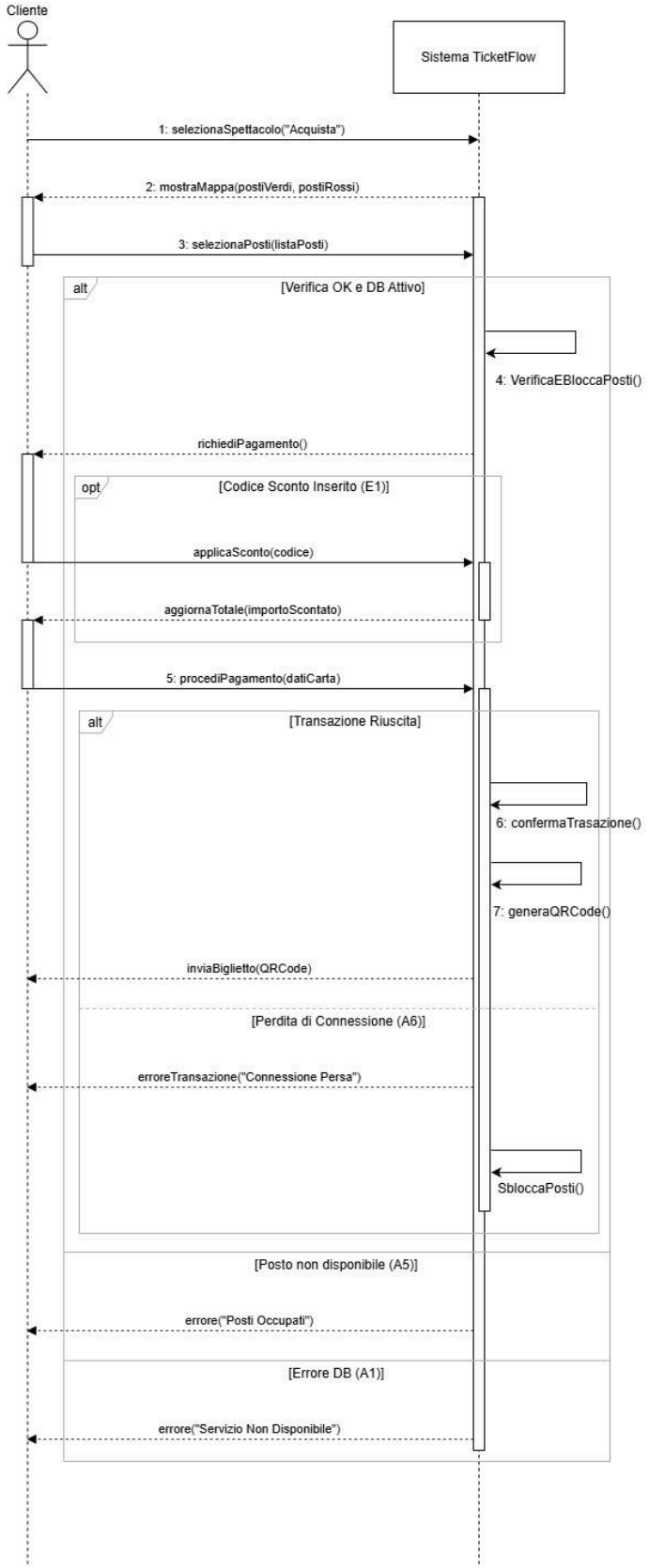
Caso d'uso 1 Cerca Spettacolo



Caso d’uso 2 Aggiungi Spettacolo

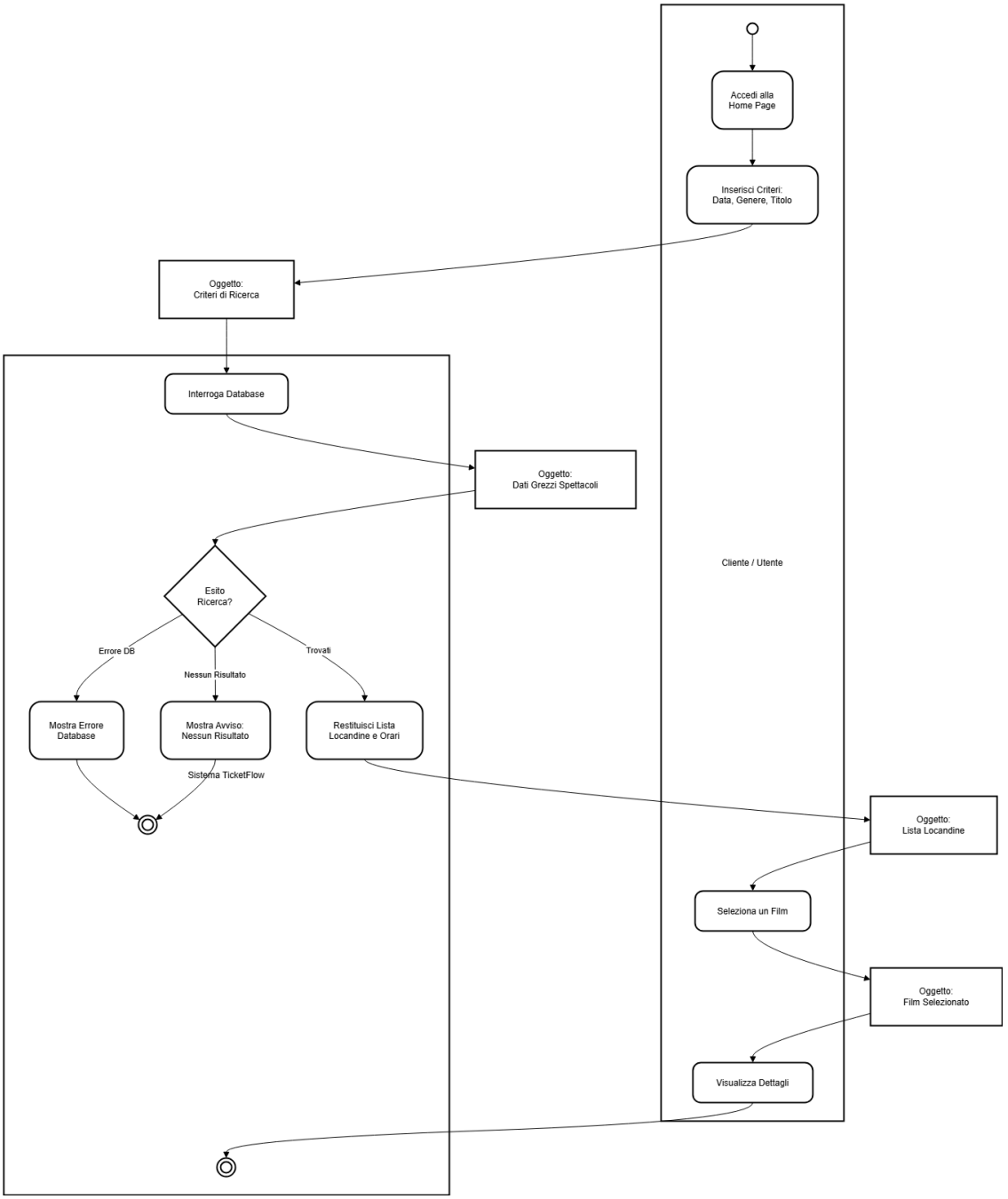


Caso d’uso 3 Acquista Biglietto

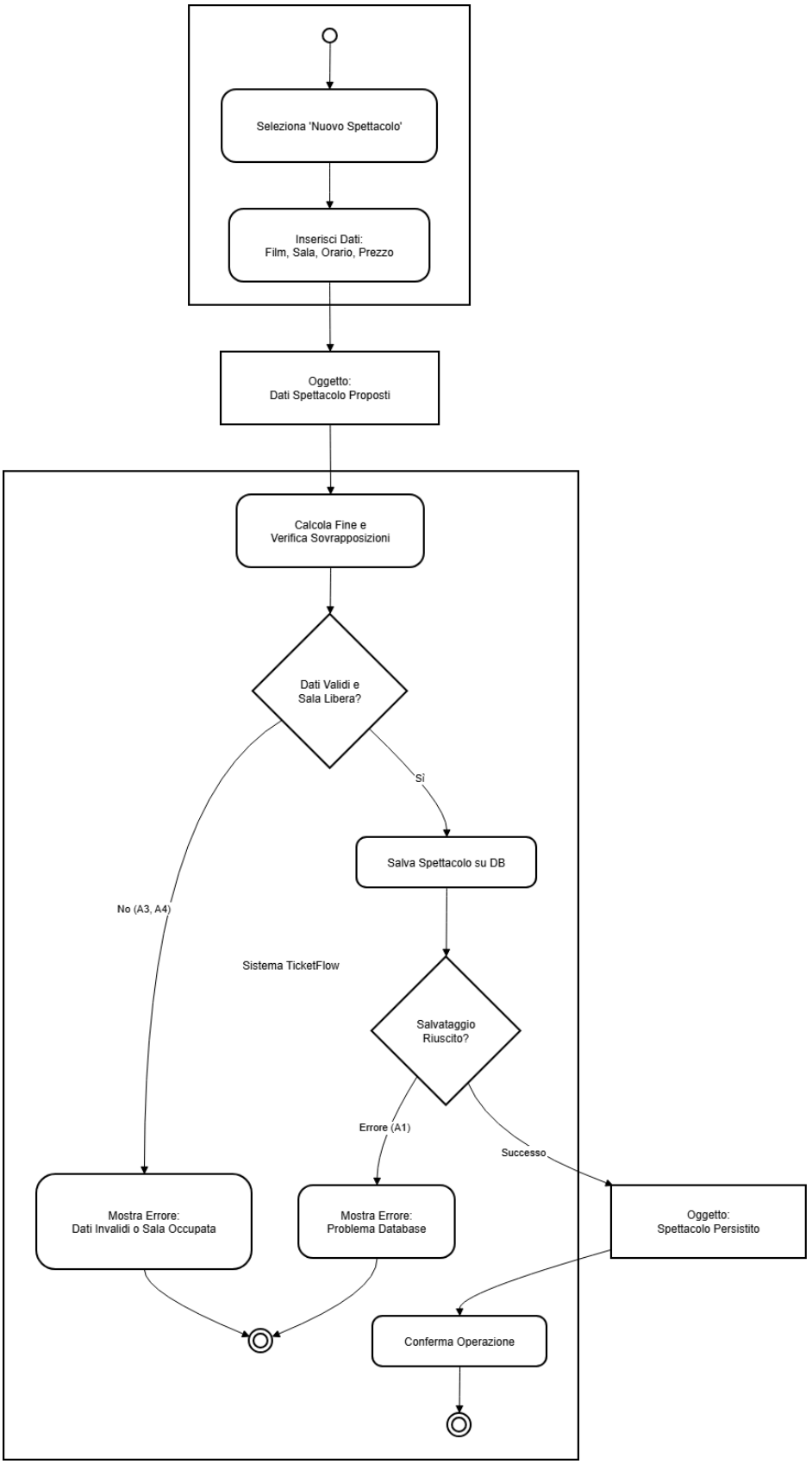
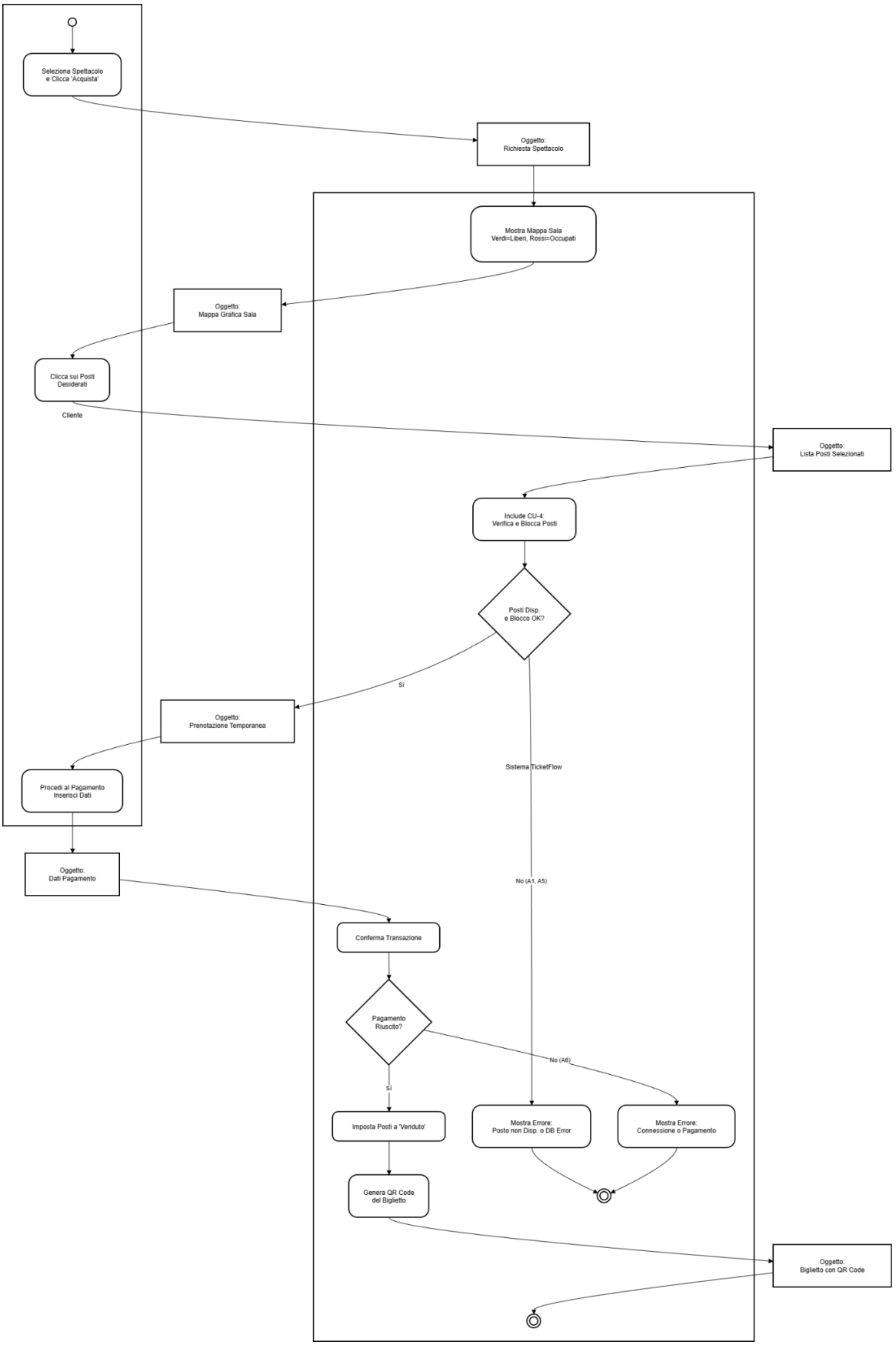


4. Diagramma di attività

Caso d’uso 1 Cerca Spettacolo



Caso d’uso 2 Aggiungi spettacolo



Caso d’uso 3 Acquisto Biglietto